

Level 3: Advanced Applied Analytics (NLP Sentiment Extraction & Ensemble Classification)

Phase Objective: Deploy Natural Language Processing workflows on text assets alongside ensemble algorithms to predict classification targets from multi-type data inputs.

1. Natural Language Processing: Textual Sentiment Extraction

Sentiment Analysis parses unstructured textual input fields to quantify subjective emotional orientation. This pipeline implements a lexicon-driven NLP methodology using the `TextBlob` library.

Algorithmic Mechanics:

Text strings are tokenized into words and cross-referenced with a lexical database containing pre-calculated sentiment weights. The primary output metric is Polarity.

```
from textblob import TextBlob

def extract_polarity(text_input):
    if not isinstance(text_input, str):
        return 0.0
    blob = TextBlob(text_input)
    return blob.sentiment.polarity

# Vectorized sentiment extraction across the Text feature series
df['Polarity_Score'] = df['ReviewText'].apply(extract_polarity)

# Stratify into discrete sentiment boundaries
df['Sentiment_Label'] = df['Polarity_Score'].apply(
    lambda score: 'Positive' if score > 0.05 else ('Negative' if score < -0.05 else 'Neutral')
)
```

Metric Definition: Polarity is bounded within $[-1.0, 1.0]$. A value of $+1.0$ indicates explicit positive sentiment, -1.0 indicates extreme negative aversion, and 0.0 signifies objective neutrality. Establishing custom boundary margins (such as ± 0.05) reduces noise from near-neutral phrases.

2. Supervised Ensemble Learning: Random Forest Classification

This pipeline builds a customer churn prediction engine using a `RandomForestClassifier`. The model processes structural features to determine whether a user profile will terminate their contract subscription (`Churn = True`).

Feature Engineering and One-Hot Encoding:

Machine learning classifiers require input features to be structured as strictly numeric arrays. Categorical text strings (such as geographic states) are transformed using One-Hot Encoding. Uninformative identifiers are dropped from the input space to prevent noise and dimensionality bloat.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Drop high-cardinality structural noise variables
df_cleaned = df.drop(['state', 'area code', 'phone number'], axis=1)

# Apply One-Hot Encoding to categorical fields, dropping the first category to prevent
df_encoded = pd.get_dummies(df_cleaned, drop_first=True)

# Separate input space X from target ground truth vector y
X = df_encoded.drop('churn_True', axis=1)
y = df_encoded['churn_True']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Technical Breakdown: `pd.get_dummies(..., drop_first=True)` converts columns with K categories into $K-1$ binary features. Dropping the first feature avoids the dummy variable trap, where perfect linear dependencies distort model assumptions. Random Forest models use Bagging (Bootstrap Aggregating) and random feature subsets to construct many uncorrelated decision trees, reducing prediction variance.

Model Verification & Diagnostics:

```
# Initialize and train the ensemble forest
rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rf_clf.fit(X_train, y_train)

# Generate inference on unseen validation subspace
y_pred = rf_clf.predict(X_test)

# Print diagnostic performance reports
print("Overall Accuracy:", accuracy_score(y_test, y_pred))
print("\nDetailed Classification Matrix:\n", classification_report(y_test, y_pred))
```

Diagnostic Interpretation: The `classification_report` tracks Precision ($\frac{TP}{TP+FP}$), Recall ($\frac{TP}{TP+FN}$), and the F_1 -score (the harmonic mean of precision and recall). This prevents deceptive accuracy evaluations on imbalanced datasets where a naive model could guess the majority class and still report high overall accuracy.

Confusion Matrix Diagnostics:

To inspect specific model errors, a Confusion Matrix is evaluated and visualized as a heatmap.

```
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Stay', 'Churn'], yti
plt.ylabel('Actual Ground Truth')
plt.xlabel('Predicted Label')
plt.title('Random Forest Confusion Matrix Heatmap')
plt.show()
```